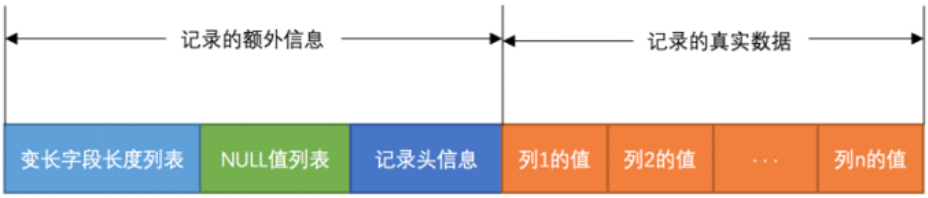
第一页：
标题：基于InnoDB的数据库系统原理（大字，放在正中间）

第二页：介绍InnoDB存储引擎（有谁开发，什么时候开发，具体是什么结构，是干什么的，
和mysql数据库的关系）

介绍词：我今天所讲的相关的数据库系统原理是基于InnoDB存储引擎的，它是一个很经典的
关系型数据库存储引擎，也是现在运用最广泛的。

第三页：InnoDB的每条信息的存储结构（标题页）

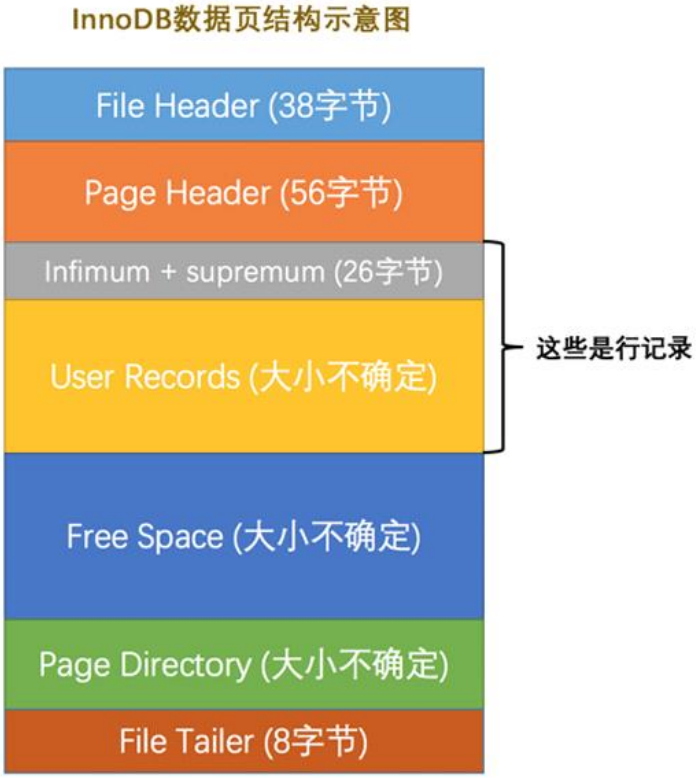
第四页：介绍InnoDB的一条记录的存储方式（以compact行格式为例）



1.介绍边长字段长度列表
2.介绍null只列表

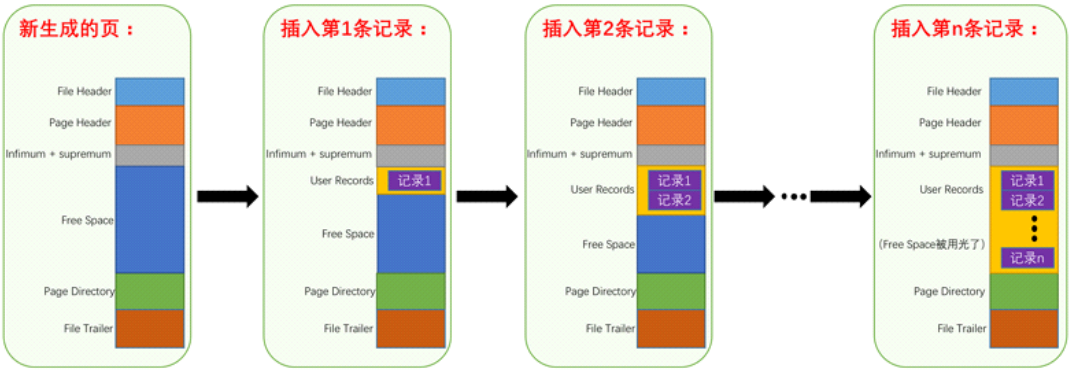
第五页：据具体例子展示compact行格式
举个具体的例子进行展示
现在有一张表中有几列
然后表中有一条数据，用这条数据举例

第六页：InnoDB的数据页结构（标题页）
第七页：InnoDB数据页结构



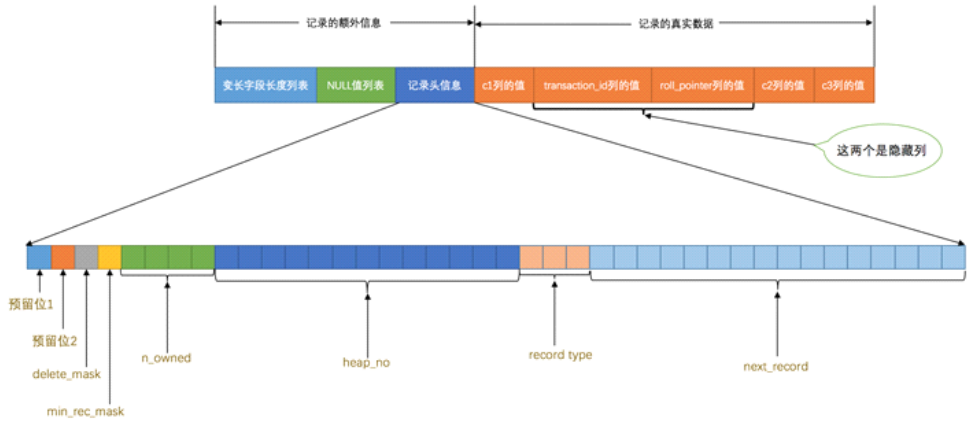
介绍记录在页中的存储方式

在页的7个组成部分中，我们自己存储的记录会按照我们指定的 行格式 存储到 User Records 部分。但是在一开始生成页的时候，其实并没有 User Records 这个部分，每当我们插入一条记录，都会从 Free Space 部分，也就是尚未使用的存储空间中申请一个记录大小的空间划分到 User Records 部分，当 Free Space 部分的空间全部被 User Records 部分替代掉之后，也就意味着这个页使用完了，如果还有新的记录插入的话，就需要去申请新的页了，这个过程的图示如下：



为了更好的管理在 User Records 中的这些记录， InnoDB 可费了一番力气呢，在哪费力气了呢？不就是把记录按照指定的行格式一条一条摆在 User Records 部分么？其实这话还得从记录行格式的 记录头信息 中说起。

介绍存储头信息：

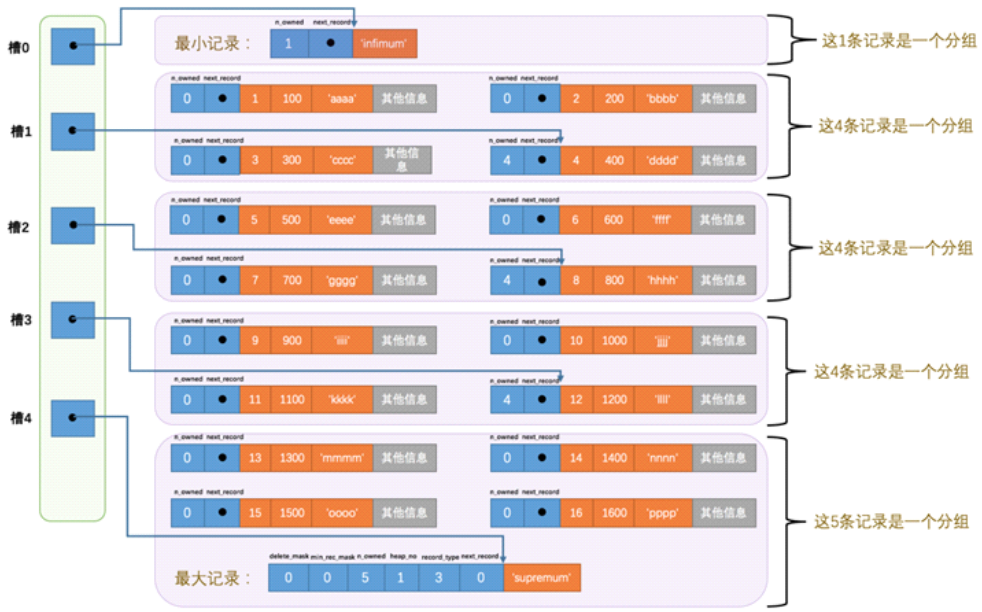


从图中可以看到，我们特意把 记录头信息 的5个字节的数据给标出来了，说明它很重要，我们再次先把这些 记录头信息 中各个属性的大体意思浏览一下（我们目前使用 Compact 行格式进行演示）：

名称	大小（单位：bit）	描述
预留位1	1	没有使用
预留位2	1	没有使用
delete_mask	1	标记该记录是否被删除
min_rec_mask	1	B+树的每层非叶子节点中的最小记录都会添加该标记
n_owned	4	表示当前记录拥有的记录数
heap_no	13	表示当前记录在记录堆的位置信息
record_type	3	表示当前记录的类型， 0 表示普通记录， 1 表示B+树非叶节点记录， 2 表示最小记录， 3 表示最大记录
next_record	16	表示下一条记录的相对位置

介绍存储头信息的每一位功能

第八页：
介绍每条记录之间的存储结构（最大记录，最小记录，页目录）

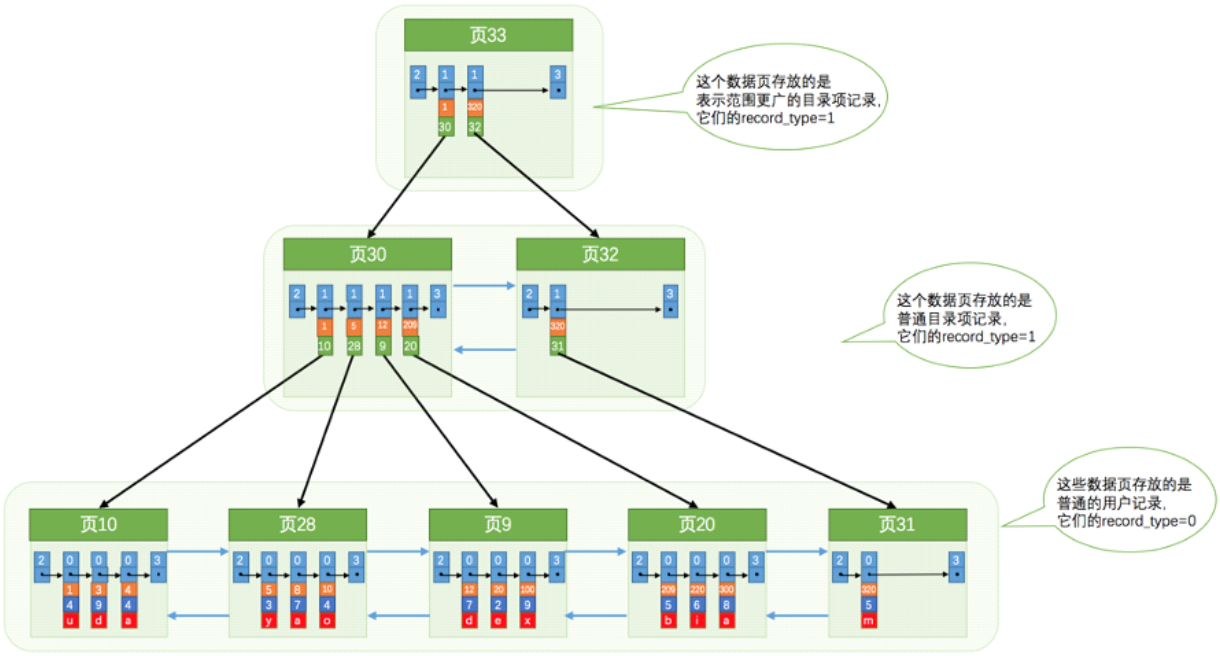


2.B+树索引

第九页：b+树索引标题页

第十页：

InnoDB的索引结构（聚簇索引，二级索引，联合索引）



第十一页：

innodb使用索引的方式：

回表的过程

3.Mysql优化策略

第十一页：mysql的优化策略标题页

第十二页;基于成本的优化

具体讲如何计算一条Sql的查询成本

第十三页：基于规则的优化

待定

4.MVCC和锁

四种问题和隔离级别

MVCC原理

锁: